Final Exam

- Time: 10:30-12:30
- Do not use "písané písmo"; write in "paličkové písmo".
- Toilet breaks are NOT allowed
- Write your answers on the provided answer sheets. Additional sheets will be supplied upon request. Please ensure that your full name is clearly written on each page of your answer sheets.
- Include detailed computation steps for all solutions. Answers without supporting calculations will receive a score of zero.

Question 1. (10 marks) For the classical ciphers discussed in this course, we assume messages consist of English letters (`A` - `Z`), where each letter is mapped to an element in $\mathbb{Z}_{26}$. Table 1 outlines the specific mapping between the letters and their corresponding elements in $\mathbb{Z}_{26}$.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
								U	V	W	X	Y	Z						
								20	21	22	23	24	25						

Table 1: Converting English letters to elements in $\mathbb{Z}_{26}$.

Recall the definition of Affine cipher as follows:

Definition 1 (Affine cipher)**.** Let $\mathcal{P} = \mathcal{C} = \mathbb{Z}_{26}$ and $\mathcal{K} = \{ (a, b) \mid a \in \mathbb{Z}_{26}^*, b \in \mathbb{Z}_{26} \}$. For each key (a, b), define

$$E_{(a,b)} : \mathbb{Z}_{26} \to \mathbb{Z}_{26}, \quad p \mapsto ap + b \mod 26; \quad D_{(a,b)} : \mathbb{Z}_{26} \to \mathbb{Z}_{26}, \quad c \mapsto a^{-1}(c - b) \mod 26.$$

The cryptosystem $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$, where $\mathcal{E} = \{ E_{(a,b)} : (a, b) \in \mathcal{K} \}$, $\mathcal{D} = \{ D_{(a,b)} : (a, b) \in \mathcal{K} \}$, is called the *affine cipher*.

a) (2 mark) How many possible keys are there for an affine cipher?

b) (2 marks) Encrypt the word `STROM` using affine cipher with the key $k = (3, 1)$.

c) (6 marks) Frequency analysis is a cryptographic technique used to break classical ciphers by studying the frequency of letters or groups of letters in a ciphertext. It exploits the fact that in any given language, certain letters or letter combinations occur more frequently than others. By comparing these frequencies in a ciphertext to the known frequency distribution of a language, an attacker can infer the likely plaintext.

   Assuming the language is English, we analyze the frequency distribution of English letters in typical English text. These frequencies are presented in Table 2.

   Consider the following ciphertext:

   `VCVIRSKPOFPNZOTHOVMLVYSATISKVNVLIVSZVR`.

   Apply frequency analysis to break the cipher and determine the original plaintext.

A	0.082	B	0.015	C	0.028	D	0.043	E	0.127	F	0.022
G	0.020	H	0.061	I	0.070	J	0.002	K	0.008	L	0.040
M	0.024	N	0.067	O	0.075	P	0.019	Q	0.001	R	0.060
S	0.063	T	0.091	U	0.028	V	0.010	W	0.023	X	0.001
Y	0.020	Z	0.001								

Table 2: Probabilities of each letter in a standard English text.

Solution.

a) (2 marks) We know that $26 = 2 \times 13$. By Theorem from lecture,

$$\varphi(26) = 26 \times \left(1 - \frac{1}{2}\right)\left(1 - \frac{1}{13}\right) = 12.$$

So there are 12 possible values for $a \in \mathbb{Z}_{26}^*$. And there are 26 possible values for $b \in \mathbb{Z}_{26}$. Then the total number of possible keys (a, b) is $12 \times 26 = 312$.

b) (2 marks) To encrypt the word `STROM`, we compute:

$$\begin{aligned} &3 \times 18 + 1 = 55 \equiv 3 \mod 26, \quad 3 \times 19 + 1 = 58 \equiv 6 \mod 26, \\ &3 \times 17 + 1 = 52 \equiv 0 \mod 26, \quad 3 \times 14 + 1 = 43 \equiv 17 \mod 26, \\ &3 \times 12 + 1 = 37 \equiv 11 \mod 26. \end{aligned}$$

So the ciphertext is `DGARL`. We can list the correspondence between plaintext and ciphertext as follows:

S	T	R	O	M
18	19	17	14	12
3	6	0	17	11
D	G	A	R	L

c) (6 marks) We can calculate the frequencies of each letter that appear in the text:

V	S	I	O	R	K	P	N	Z	T	L	C	F	H	M	Y	A
8	4	3	3	2	2	2	2	2	2	2	1	1	1	1	1	1

The most frequent letter is `V`, and the second most frequent one is `S`. As a first frequency-analysis guess, we assume that `V` corresponds to `E` and `S` corresponds to `T`. (1 mark)

Let the key be (a, b). By Definition, we have the following equations:

$$4a + b = 21 \mod 26, \quad 19a + b = 18 \mod 26,$$

which gives

$$15a = 23 \mod 26.$$

By the extended Euclidean algorithm,

$$26 = 15 \times 1 + 11, \quad 15 = 11 \times 1 + 4, \quad 11 = 4 \times 2 + 3, \quad 4 = 3 + 1,$$

and

$$\begin{aligned} 1 &= 4 - 3 = 4 - (11 - 4 \times 2) = -11 + 4 \times 3 = -11 + (15 - 11) \times 3 \\ &= 15 \times 3 - 11 \times 4 = 15 \times 3 - (26 - 15) \times 4 = 15 \times 7 - 26 \times 4. \end{aligned}$$

Hence, we have $15^{-1} \mod 26 = 7$ and

$$a = 23 \times 15^{-1} \mod 26 = 23 \times 7 \mod 26 = 5 \mod 26.$$

(2 marks)

Furthermore, we get

$$b = 21 - 4a \mod 26 = 21 - 4 \times 5 \mod 26 = 1.$$

(1 mark)

To decrypt the message, we compute the decryption key by finding $a^{-1} \mod 26 = 5^{-1} \mod 26$:

$$26 = 5 \times 5 + 1 \Longrightarrow 1 = 26 - 5 \times 5 \Longrightarrow 5^{-1} \mod 26 = -5 \mod 26 = 21.$$

(1 mark)

Using $a^{-1} = 21$ and $b = 1$, we decrypt via

$$p = 21(c - 1) \mod 26.$$

This gives the plaintext

`EVERYTHING IS KNOWN EXCEPT FOR THE SECRET KEY.`

(1 mark)

Question 2. (10 marks) In this question, we consider RSA encryption and decryption. Suppose Bob would like to generate his public and private keys for RSA. Bob randomly generates $p = 29$ and $q = 41$.

a) (1 mark) Then he computes

$$n = ? \quad \varphi(n) = ?$$

b) (2 marks) From $\mathbb{Z}^*_{\varphi(n)}$, Bob chooses his public exponent $e = 3$. Compute, using the extended Euclidean algorithm, Bob's private key d.

c) (1 mark) Alice would like to send plaintext $m = 2$ to Bob, using Bob's public key n, e. Alice computes ciphertext

$$c = ?$$

d) (3 marks) Bob receives a message $c = 142$ from Alice. Then to decrypt the message $c = 142$ with CRT-based RSA, Bob computes

$$m_p = ? \quad m_q = ? \quad y_p = ? \quad y_q = ?$$

Using Garner's algorithm, Bob gets

$$m = ?$$

e) (3 marks) Now we consider a malicious attacker who carries out a Bellcore attack during the RSA decryption. Suppose she injects fault during the computation of m_p and the faulty $m'_p = 1$. Then the faulty message will be

$$m' = ?$$

With the knowledge of m, m' and the public key n, e, how can the attacker recover the secret key d?

Solution.

a) (1 mark)

$$n = 29 \times 41 = 1189, \quad \varphi(n) = 28 \times 40 = 1120.$$

b) (2 marks) By the extended Euclidean algorithm

$$1120 = 3 \times 373 + 1 \Longrightarrow 1 = 1120 - 3 \times 373.$$

Hence Bob's private key $d = -373 \mod 1120 = 747$.

c) (1 mark) Alice computes

$$c = m^e \mod n = 2^3 \mod 1189 = 8.$$

d) (3 marks) After receiving the ciphertext $c = 142$, with CRT-based RSA implementation, Bob computes

$$\begin{aligned} m_p &= c^{d \mod (p-1)} \mod p = 142^{747 \mod 28} \mod 29 = 26^{19} \mod 29 = 11, \text{(1 mark)} \\ m_q &= c^{d \mod (q-1)} \mod q = 142^{747 \mod 40} \mod 41 = 19^{27} \mod 41 = 11. \text{(1 mark)} \end{aligned}$$

To compute $26^{19} \mod 29$, we note that

$$\begin{aligned} 26^2 \mod 29 &= 9, \\ 26^4 \mod 29 &= 9^2 \mod 29 = 23, \\ 26^8 \mod 29 &= 23^2 \mod 29 = 7, \\ 26^{16} \mod 29 &= 7^2 \mod 29 = 20. \end{aligned}$$

Thus

$$26^{19} \mod 29 = 26^{16} \times 26^2 \times 26 \mod 29 = 20 \times 9 \times 26 \mod 29 = 11.$$

Similarly,

$$\begin{aligned} 19^2 \mod 41 &= 33 \\ 19^3 \mod 41 &= 33 \times 19 \mod 41 = 12 \\ 19^9 \mod 41 &= 12^3 \mod 41 = 6 \\ 19^{27} \mod 41 &= 6^3 \mod 41 = 11. \end{aligned}$$

By the extended Euclidean algorithm

$$41 = 29 + 12, \ 29 = 12 \times 2 + 5, \ 12 = 5 \times 2 + 2, \ 5 = 2 \times 2 + 1,$$

which gives

$$\begin{aligned} 1 &= 5 - 2 \times (12 - 5 \times 2) = -2 \times 12 + (29 - 12 \times 2) \times 5 \\ &= 29 \times 5 - 12 \times (41 - 29) = -41 \times 12 + 29 \times 17. \end{aligned}$$

We have (0.5 marks)

$$\begin{aligned} y_p &= p^{-1} \mod q = 29^{-1} \mod 41 = 17 \mod 41, \\ y_q &= q^{-1} \mod p = 41^{-1} \mod 29 = -12 \mod 29 = 17 \mod 29. \end{aligned}$$

By Garner's algorithm, (0.5 marks)

$$m = m_p + ((m_q - m_p)y_p \mod q)p = 11 + 0 = 11$$

e) (3 marks) The faulty message is given by

$$\begin{aligned} m' &= m'_p + ((m_q - m'_p)y_p \mod q)p = 1 + ((11-1)\times 17 \mod 41)\times 29 \\ &= 1 + 6\times 29 = 175. \end{aligned}$$

According to the Bellcore attack,

$$q = \gcd(m' - m, n) = \gcd(175 - 11, 1189) = \gcd(164, 1189).$$

By the Euclidean algorithm

$$\begin{aligned} 1189 &= 164\times 7 + 41, & \gcd(164, 1189) &= \gcd(164, 41), \\ 164 &= 41\times 4, & \gcd(164, 41) &= 41. \end{aligned}$$

Hence $q = 41$ and

$$p = \frac{n}{q} = 29.$$

Then the attacker can compute

$$\varphi(n) = (p-1)(q-1) = 1120.$$

The private key

$$d = e^{-1} \mod \varphi(n) = 3^{-1} \mod 1120.$$

can be found using the extended Euclidean algorithm.

Question 3. (5 marks) The stochastic leakage model assumes each bit of the target intermediate value $\boldsymbol{v} = v_{m_v-1}v_{m_v-2}\dots v_1v_0$ has a different leakage.

$$\mathcal{L}(\boldsymbol{v}) = \sum_{s=0}^{m_v-1} \alpha_s v_s + \text{noise},$$

where noise $\sim \mathcal{N}(0, \sigma^2)$ denotes the noise with variance σ^2 and α_s $(s = 0, 1, \dots, m_v - 1)$ are real numbers. We have discussed how to profile the DUT to find estimations for α_s. Let

$$\boldsymbol{\ell}_{pf} := (l^{1,pf}_{\text{POI}}, l^{2,pf}_{\text{POI}}, \dots, l^{M_{pf},pf}_{\text{POI}})$$

be the vector of leakages at t =POI from all M_{pf} profiling traces, where $l^{j,pf}_{\text{POI}}$ is the leakage at POI from the jth profiling trace.

Furthermore, for the jth profiling trace, let

$$\boldsymbol{v}^{pf}_j = v^{pf}_{j(m_v-1)}\dots v^{pf}_{j1}v^{pf}_{j0}, \quad j = 1, 2, \dots, M_{pf}$$

be the corresponding target intermediate value. Then we compute matrix $M_{\boldsymbol{v}}$

$$M_{\boldsymbol{v}} := \begin{pmatrix} v^{pf}_{10} & v^{pf}_{11} & \dots & v^{pf}_{1(m_v-1)} \\ v^{pf}_{20} & v^{pf}_{21} & \dots & v^{pf}_{2(m_v-1)} \\ \vdots & \vdots & \ddots & \vdots \\ v^{pf}_{M_{pf}0} & v^{pf}_{M_{pf}1} & \dots & v^{pf}_{M_{pf}(m_v-1)} \end{pmatrix}$$

The estimated values $\hat{\alpha}_s$ for α_s are given by

$$(\hat{\alpha}_0, \hat{\alpha}_1, \dots, \hat{\alpha}_{m_v-1}) = \left(M^T_{\boldsymbol{v}} M_{\boldsymbol{v}}\right)^{-1} M^T_{\boldsymbol{v}} \boldsymbol{\ell}_{pf}.$$

Suppose we are interested in the computation of the first round of PRESENT. Our target intermediate value $\boldsymbol{v}$ is the 0th Sbox output in the first round. We have collected $M_{pf} = 5000$ traces.

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
C	5	6	B	9	0	A	D	3	E	F	8	4	7	1	2

Table 3: PRESENT Sbox.

a) (2 marks) The first trace in our dataset corresponds to the 0th nibble of the plaintext= 4 and the 0th nibble of the first round key= 7. Thus the intermediate value for the first trace is given by:

$$\boldsymbol{v}_1^{pf} = ?$$

b) (1 mark) The first row of our matrix $M_{\boldsymbol{v}}$ is given by?

c) (1 mark) Suppose we got the following estimated values for α_ss:

$$\hat{\alpha}_0 \approx -0.02019, \quad \hat{\alpha}_1 \approx -0.02027, \quad \hat{\alpha}_2 \approx -0.01920, \quad \hat{\alpha}_3 \approx -0.02039.$$

Then according to the stochastic leakage model, the leakage of $\boldsymbol{v} = v_3v_2v_1v_0$ is given by

$$\mathcal{L}(\boldsymbol{v}) = ?.$$

d) (1 mark) In particular,

$$\mathcal{L}(\mathtt{E}) = ?$$

Solution.

a) (2 marks) The first trace in our dataset corresponds to the 0th nibble of the plaintext= 4 and the 0th nibble of the first round key= 7. Thus the intermediate value for the first trace is given by:

$$\boldsymbol{v}_1^{pf} = \mathrm{SB}_{\mathrm{PRESENT}}(4 \oplus 7) = \mathrm{SB}_{\mathrm{PRESENT}}(3) = \mathtt{B} = 1011_2.$$

b) (1 mark) And the first row of our matrix $M_{\boldsymbol{v}}$ is given by

$$\begin{pmatrix} 1 & 1 & 0 & 1 \end{pmatrix}.$$

c) (1 mark) For $\boldsymbol{v} = v_3v_2v_1v_0$, the leakage is given by

$$\mathcal{L}(\boldsymbol{v}) = \hat{\alpha}_0 v_0 + \hat{\alpha}_1 v_1 + \hat{\alpha}_2 v_2 + \hat{\alpha}_3 v_3 + \text{noise}.$$

d) (1 mark) $\mathcal{L}(\mathtt{E}) = \hat{\alpha}_1 + \hat{\alpha}_2 + \hat{\alpha}_3 + \text{noise} = -0.05986 + \text{noise}.$

Question 4. (5 marks) We have introduced a method for implementing masked PRESENT by using precomputed lookup tables, denoted as T1 and T2. For any $\boldsymbol{v} \in \mathbb{F}_2^4$, any input mask $\mathfrak{m}_{\text{in}} \in \mathbb{F}_2^4$ and the corresponding output mask $\mathfrak{m}_{\text{out}} \in \mathbb{F}_2^4$ for PRESENT Sbox, the table T1 is defined as:

$$\mathrm{T1}[\boldsymbol{v} \oplus \mathfrak{m}_{\text{in}}, \mathfrak{m}_{\text{in}}] = \mathrm{SB}(\boldsymbol{v}) \oplus \mathfrak{m}_{\text{out}}. \tag{1}$$

Additionally, T2 is used to keep track of the masks:

$$\mathrm{T2}[\mathfrak{m}_{\text{in}}] = \mathfrak{m}_{\text{out}}, \quad \mathfrak{m}_{\text{in}} = \mathtt{0}, \mathtt{1}, \ldots, \mathtt{F}. \tag{2}$$

This approach eliminates the need to regenerate a masked S-box table each time the input mask changes.

Please complete the masked PRESENT algorithm by filling in each missing line based on the specific instructions provided.

a) For Line 7, implement the computation of the masked Sbox.

b) For line 8, store the resulting output masks from the Sbox operation.

c) For line 9, apply the pLayer operation to the current cipher state.

d) For line 10, apply the pLayer operation to the masks.

e) For line 12, remove masks from the ciphertext.

Algorithm 1: Masked implementation of PRESENT.

Input: p, T1, T2, K_i $(i = 1, 2, \ldots, 32)$// p is the plaintext for encryption; T1 is the table for masked Sbox as given in Equation 1; T2 specifies the output mask given the input mask for PRESENT Sbox as defined in Equation 2; K_i are round keys for PRESENT encryption

Output: ciphertext

1 randomly generate 16 masks $\mathfrak{m}_0, \mathfrak{m}_1, \ldots, \mathfrak{m}_{15}$
2 **array** of size 16 state $= p \oplus \mathfrak{m}_{15}, \mathfrak{m}_{14}, \ldots, \mathfrak{m}_1, \mathfrak{m}_0$// mask the jth nibble of the plaintext with $\mathfrak{m}_j$, each entry of the array is one masked nibble
3 **array** of size 16 masks $= \mathfrak{m}_{15}, \mathfrak{m}_{14}, \ldots, \mathfrak{m}_1, \mathfrak{m}_0$
4 **for** $i = 0$, $i < 31$, $i++$ **do**
5 state = addRoundKey(state, K_i)
6 **for** $j = 0$, $j < 16$, $j++$ **do**
 // for each nibble
7 ?// masked Sbox computation
8 ?// record the output masks of Sbox computation
9 ?// apply pLayer to the cipher state
10 ?// apply pLayer to the masks
11 state = addRoundKey(state, K_i)
12 ?// remove masks from the ciphertext
13 **return** state

Solution.

Algorithm 2: Masked implementation of PRESENT.

Input: p, T1, T2, K_i $(i = 1, 2, \ldots, 32)$// p is the plaintext for encryption; T1 is the table for masked Sbox as given in Equation 1; T2 specifies the output mask given the input mask for PRESENT Sbox as defined in Equation 2; K_i are round keys for PRESENT encryption

Output: ciphertext

```
randomly generate 16 masks    m_0, m_1, ..., m_15
array of size 16    state = p ⊕ m_15, m_14, ..., m_1, m_0// mask the jth nibble of the
    plaintext with m_j, each entry of the array is one masked nibble
array of size 16    masks = m_15, m_14, ..., m_1, m_0
for i = 0, i < 31, i++ do
    state = addRoundKey(state, K_i)
    for j = 0, j < 16, j++ do
        // for each nibble
        state[j] = T1[state[j], masks[j]]// masked Sbox computation
        masks[j] = T2[masks[j]]// record the output masks of Sbox computation
    state = pLayer(state)// apply pLayer to the cipher state
    masks = pLayer(masks)// apply pLayer to the masks
state = addRoundKey(state, K_i)
state = state ⊕ masks
return state
```

Question 5. (10 marks) For an Sbox $\mathrm{SB} : \mathbb{F}_2^{\omega_1} \to \mathbb{F}_2^{\omega_2}$, the *(extended) difference distribution table (DDT)* of SB is a 2-dimensional table T of size $2^{\omega_2} \times (2^{\omega_1} - 1)$ such that for any $0 < \delta < 2^{\omega_1}$ and $0 \leq \Delta < 2^{\omega_2}$, the entry of T at the Δth row and δth column is given by

$$T[\Delta, \delta] = \{\, \boldsymbol{a} \mid \boldsymbol{a} \in \mathbb{F}_2^{\omega_1},\ \mathrm{SB}(\boldsymbol{a} \oplus \delta) \oplus \mathrm{SB}(\boldsymbol{a}) = \Delta \,\}.$$

Here we identify elements of $\mathbb{F}_2^{\omega_1}$ and $\mathbb{F}_2^{\omega_2}$ with their binary representations as integers when indexing the rows and columns of the table. We refer to δ as the *input difference*, and Δ as the *output difference*.

Let SB be the following Sbox

x	0	1	2	3	4	5	6	7
$\mathrm{SB}(x)$	4	7	0	5	2	6	3	1

Define

$$\begin{aligned} f : \mathbb{F}_2^3 &\to \mathbb{F}_2^3 \\ \boldsymbol{x} &\mapsto \mathrm{SB}(\boldsymbol{x} \oplus \boldsymbol{a}) \oplus \boldsymbol{b}. \end{aligned}$$

We consider a differential fault analysis attack on f. Our attack assumption is as follows:

- Fault location: input of f
- Fault model: bit flip
- Fault mask: $\varepsilon \in \mathbb{F}_2^3$ s.t. $\boldsymbol{x}' = \boldsymbol{x} \oplus \varepsilon$
- Attacker knowledge: Sbox design, inputs and outputs of f, fault mask

- Attacker goal: recover values of $\boldsymbol{a}$ and $\boldsymbol{b}$
- The Attacker can repeat the computation with the same input (not chosen by the attacker)

We know that with input $\boldsymbol{x} = 3$, the correct output is 1.

a) (4 marks) Complete the DDT for SB

Δ \ δ	1	2	3	4	5	6	7
1		46	03	15	27		
2	67	13			05	24	
3	01		47	26		35	
4	45	02		37			16
5	23		56		14		07
6				04	36	?	?
7		57	12			?	?

b) (2 marks) Suppose the attacker injects one fault with fault mask $\varepsilon_1 = 2$ and the resulting output is 5. What do we know about $\boldsymbol{a}$?

c) (4 marks) Suppose the attacker injects another fault with fault mask $\varepsilon_2 = 3$ and the resulting output is 0. Find the values of $\boldsymbol{a}$ and $\boldsymbol{b}$.

Solution.

a) (4 marks)

Δ \ δ	1	2	3	4	5	6	7
1		46	03	15	27		
2	67	13			05	24	
3	01		47	26		35	
4	45	02		37			16
5	23		56		14		07
6				04	36	17	25
7		57	12			06	34

b) (2 marks) For a fault mask ε, let Δ denote the difference between the correct and faulty output, then

$$\begin{aligned}\Delta &= (\mathrm{SB}(\boldsymbol{x} \oplus \boldsymbol{a}) \oplus \boldsymbol{b}) \oplus (\mathrm{SB}(\boldsymbol{x}' \oplus \boldsymbol{a}) \oplus \boldsymbol{b}) = \mathrm{SB}(\boldsymbol{x} \oplus \boldsymbol{a}) \oplus \mathrm{SB}(\boldsymbol{x}' \oplus \boldsymbol{a}) \\ &= \mathrm{SB}(\boldsymbol{x} \oplus \boldsymbol{a}) \oplus \mathrm{SB}(\boldsymbol{x} \oplus \boldsymbol{a} \oplus \varepsilon).\end{aligned}$$

We can conclude that he value $\boldsymbol{x} \oplus \boldsymbol{a}$ is in the entry of DDT corresponding to input difference $\delta = \varepsilon$ and output difference Δ.
With fault mask 2 and

$$\Delta = 1 \oplus 5 = 4,$$

we know that the value $\boldsymbol{x} \oplus \boldsymbol{a}$ is in the entry of DDT corresponding to input difference 2 and output difference 4. Thus the possible values of $\boldsymbol{x} \oplus \boldsymbol{a}$ are given by 0 and 2. Knowing that $\boldsymbol{x} = 3$, the possible values of $\boldsymbol{a}$ are 3 and 1.

c) (4 marks) Similarly, with fault mask $\mathtt{3}$ and

$$\Delta = \mathtt{1} \oplus \mathtt{0} = \mathtt{1},$$

we know that the value $\boldsymbol{x} \oplus \boldsymbol{a}$ is in the entry of DDT corresponding to input difference $\mathtt{3}$ and output difference $\mathtt{1}$. Thus the possible values of $\boldsymbol{x} \oplus \boldsymbol{a}$ are given by $\mathtt{0}$ and $\mathtt{3}$. Knowing that $\boldsymbol{x} = \mathtt{3}$, the possible values of $\boldsymbol{a}$ are $\mathtt{3}$ and $\mathtt{0}$. Together with the previous answer, we know that the value of $\boldsymbol{a} = \mathtt{3}$.

We also know that when $\boldsymbol{x} = \mathtt{3}$,

$$\mathrm{SB}(\mathtt{3} \oplus \boldsymbol{a}) \oplus \boldsymbol{b} = \mathtt{1},$$

which gives

$$\boldsymbol{b} = \mathrm{SB}(\mathtt{3} \oplus \mathtt{3}) \oplus \mathtt{1} = \mathrm{SB}(\mathtt{0}) \oplus \mathtt{1} = \mathtt{4} \oplus \mathtt{1} = \mathtt{5}.$$